# Git & GitHub Complete Guide (Beginner to Intermediate)

*For Flutter Developers (VS Code & Android Studio)*

---

## 1. Version Control System (VCS)

### Version Control কি?

Version Control System এমন একটি system যা একটি project এর code এর পরিবর্তনের ইতিহাস সংরক্ষণ করে। অর্থাৎ, আপনি যদি code পরিবর্তন করেন, delete করেন বা নতুন feature যোগ করেন সবকিছু history হিসেবে save থাকে। তাই প্রয়োজন হলে আগের version এ ফিরে যাওয়া যায়।

### Example

ধরুন একটি Flutter project এ কাজ করছেন।

Day-1

```
Login Screen Added
```

Day-2

```
Bug Fix
```

Day-3

```
UI Updated
```

Day-4 এ যদি UI update এ সমস্যা হয় তাহলে আপনি Day-2 এর code এ ফিরে যেতে পারবেন।

### Version Control কেন ব্যবহার করা হয়?

1. Code history track করার জন্য
2. Team collaboration এর জন্য
3. Bug হলে আগের version restore করার জন্য
4. Multiple developer একসাথে কাজ করার জন্য
5. Backup হিসেবে

### Version Control Diagram

```
Version1 → Version2 → Version3 → Version4
Initial    Bug Fix    Feature    UI Update
```

প্রতিটি version কে Git এ বলা হয় commit.

## 2. Git কি?

Git হলো একটি Distributed Version Control System. এটি developer এর machine এ project এর history track করে।

Git ব্যবহার করে আপনি:

- code history track করতে পারবেন
- branch তৈরি করতে পারবেন
- merge করতে পারবেন
- previous version restore করতে পারবেন

## 3. GitHub কি?

GitHub হলো একটি online platform যেখানে Git repository host করা হয়।

GitHub ব্যবহার করে:

- code online এ রাখা যায়
- team collaboration করা যায়
- remote backup রাখা যায়

### Git vs GitHub

| Git | GitHub |
|---|---|
| Local version control | Online code hosting |
| Local machine এ কাজ করে | Cloud platform |
| Command line tool | Web platform |

# 4. Git Installation

Git install করতে হবে।

Website:

https://git-scm.com

Download → Install → Next → Finish

## Git Install Verify

Run this command in terminal:

```
git -version
```

Example Output

```
git version 2.44.0
```

# 5. Git Configuration

## Git Config কেন ব্যবহার করা হয়?

Git commit করার সময় জানতে চায়:

- কে commit করছে
- কোন email দিয়ে commit করছে

এই তথ্য commit history তে সংরক্ষিত থাকে। তাই Git configuration ব্যবহার করে *username* ও *email* set করা হয়।

## Username Set

```
git config --global user.name "Your Name"
```

Example

```
git config --global user.name "Shaon"
```

### Email Set

```
git config --global user.email "your@email.com"
```

Example

```
git config --global user.email shaonnubtk22@gmail.com
```

### Configuration Check

```
git config -list
```

## 6. Git Repository

### Repository কি?

Repository হলো project এর এমন একটি folder যেখানে Git history track করে।

### git init কেন ব্যবহার করা হয়?

git init command একটি project folder কে Git repository তে convert করে। এটি `.git` নামে একটি hidden folder তৈরি করে যেখানে history store হয়।

### Command

```
git init
```

### Diagram

```
Project Folder
      |
      | git init
      ↓
Project + .git folder
```

# 7. Git Workflow (Basic Flow)

Git এ কাজ করার basic flow:

```
Working Directory
        |
        | git add
        ↓
Staging Area
        |
        | git commit
        ↓
Local Repository
        |
        | git push
        ↓
Remote Repository (GitHub)
```

# 8. Git Status

Project এর current state দেখার জন্য।

```
git status
```

এটি দেখায়:

- modified files
- staged files
- untracked files

# 9. Git Staging

Staging কেন ব্যবহার করা হয়?

Git এ commit করার আগে developer decide করতে পারে কোন file commit হবে। এটি staging area তে রাখা হয়।

## Specific File Stage

```
git add file_name.dart
```

## All Files Stage

```
git add .
```

## Diagram

```
File Edited
    |
    | git add
    ↓
Staging Area
```

# 10. Git Commit

## Commit কি?

Commit হলো code এর snapshot. এটি project এর একটি version save করে।

## Commit Command

```
git commit -m "Login screen added"
```

## Commit History Diagram

```
Commit 1
Initial Project

↓

Commit 2
Login Screen

↓

Commit 3
Bug Fix
```

# 11. GitHub Repository

GitHub এ repository create করতে হবে। (যদি github account না থাকে তাহলে https://github.com/ এই URL থেকে github accout create করতে হবে)

Example (Run this command in terminal)

```
flutter_login_app
```

# 12. Git Remote

Remote repository connect করার জন্য (username এর স্থানে Github username ব্যবহার করতে হবে)

```
git remote add origin https://github.com/username/repository.git
```

# 13. .gitignore

.gitignore কি?

.gitignore হলো একটি special file যা Git কে বলে কোন files/folders track বা commit করতে হবে না।

.gitignore কেন ব্যবহার করা হয়?

- Build files / Generated files ignore করা
  যেমন Flutter এর build/ folder, Android Studio এর *.iml files
- Sensitive data hide করা
  যেমন API keys, passwords, .env files
- Unnecessary files avoid করা
  যেমন logs, cache, temporary files

Example of .gitignore for Flutter

```
# Flutter/Dart build files
build/
.flutter-plugins
```

```
.flutter-plugins-dependencies
.dart_tool/
.packages

# IntelliJ/Android Studio files
*.iml
*.ipr
*.iws
.idea/

# VS Code files
.vscode/

# Logs
*.log

# MacOS files
.DS_Store
```

কোথায় add করতে হবে?

.gitignore file project এর root folder এ রাখতে হবে। Android Studio এর ক্ষেত্রে .gitignore file project create করার সময় auto generate হয়।

Flutter project এর ক্ষেত্রে:

my_flutter_app/
├──── android/
├──── ios/
├──── lib/
├──── build/
├──── .gitignore ← এখানে রাখতে হবে
├──── pubspec.yaml
└──── ...

## Usage

- .gitignore create করুন project root এ
- Ignore করতে চাওয়া files/folders এর নাম লিখুন
- Git automatically এই files track করবে না

## Example:

```
git status
```

এখন .gitignore এ থাকা files Git status এ দেখাবে না।

# 14. Git Push

Git Push কেন ব্যবহার করা হয়?

Local repository এর commits GitHub এ upload করার জন্য।

## Command

```
git push -u origin main
```

## Explanation

```
push = upload code
origin = remote repository
main = branch name
-u = upstream set
```

# 15. Git Pull

Remote repository থেকে latest code download করে।

```
git pull origin main
```

# 16. Git Fetch

Fetch remote repository থেকে changes download করে merge না করে।

```
git fetch
```

# 17. Branching

Branch ব্যবহার করে নতুন feature develop করা যায়।

## Branch Diagram

```
main
 |
 |
 |------ login-feature
 |
 |------ payment-feature
```

## Create Branch

```
git branch feature-login
```

## Switch Branch

```
git checkout feature-login
```

# 18. Git Merge

## Merge কি?

একটি branch এর changes অন্য branch এ যোগ করা।

## Command

```
git merge feature-login
```

## Diagram

```
main
 |
 |----- feature-login
 |
 | merge
 ↓
main (login feature added)
```

# 19. Merge Conflict

## Conflict কেন হয়?

যখন দুই developer একই file এর একই line change করে।

## Conflict Example

Developer A

```
color = red
```

Developer B

```
color = blue
```

Git বুঝতে পারে না কোনটি রাখবে।

## Conflict File

```
<<<<<<< HEAD
color = red
=======
color = blue
>>>>>>> feature
```

Developer manually fix করে।

# 20. Git Rebase

Rebase commit history clean করার জন্য ব্যবহার করা হয়।

## Merge vs Rebase Diagram

### Actual

```
A---B---C main
     \
      D---E feature
```

Merge করলে

```
A---B---C-----F
     \       /
      D-----E
```

Rebase

```
A---B---C main
         \
          D---E feature
```

History linear হয়।

# 21. Git Stash

## Stash কেন ব্যবহার করা হয়?

যখন incomplete code আছে কিন্তু branch change করতে হবে। Stash temporary save করে।

## Command

```
git stash
```

## Restore

```
git stash pop
```

## Diagram

```
Working Code
    |
git stash
    ↓
Temporary Storage
```

# 22. Git Reset

Reset commit history change করতে ব্যবহার করা হয়।

## Soft Reset

```
git reset --soft HEAD~1
```

Last commit remove কিন্তু code থাকে।

## Hard Reset

```
git reset --hard HEAD~1
```

Commit + code remove

# 23. Git Revert

Revert কেন ব্যবহার করা হয়?

Commit history delete না করে undo commit তৈরি করার জন্য।

## Command

```
git revert commit_id
```

# 24. Commit History থেকে Code Restore

Commit History দেখার জন্য

```
git log
```

Output

```
commit 34f6a1
commit 23a5b9
commit 98c1d2
```

### Specific Commit Checkout

```
git checkout commit_i
```

### Specific Commit Code Apply

```
git cherry-pick commit_id
```

## 25. Flutter Developer Workflow

VS Code / Android Studio terminal এ কাজ।

### Workflow

```
flutter create app

git init

git add .

git commit -m "Initial commit"

git remote add origin repo_link

git push -u origin main
```

## 26. Complete Git Workflow Diagram

```
Code Write
     |
git add
     |
git commit
     |
Local Repository
     |
git push
     |
GitHub Repository
     |
Team Pull
```